

For an isolated, autonomous **Red Agent** acting as an adversarial gatekeeper in a mesh topology, standard file management and chat workflows will not work. It requires a strict, fail-closed design that prevents context bleeding, accidental command leakage, and state pollution.

The optimal file management layout and CLI configuration for the Red Agent node isolate it completely from direct user interaction, forcing all workflows through an immutable, auditable structure.

1. The Red Agent File System Layout (/opt/red-agent/)

The workspace must be compartmentalized so the auditing model can only read generated action logs and write to an append-only verdict queue.

Plaintext

/opt/red-agent/

├── config/

│ ├── target_rules.yaml # Strict rules of engagement / boundary blocks

│ └── audit_matrix.json # Criteria for JSONL schema violation & hallucination checks

├── incoming/

│ └── pending_batch.jsonl # Staged action logs dropped by the UI/Scripting Hub

├── audit/

│ ├── approved/ # Cleaned JSONL ready for cloud recursive KAG training

│ └── quarantined/ # Flagged logs containing malformed data, logic loops, or

injection

├── skills/

│ ├── SKILL-audit-jsonl.md # Action-specific instructions for the auditing model

│ └── SKILL-validate-schema.md # Deterministic parsing rules

└── state/

└── audit_ledger.db # Local SQLite database tracking pass/fail metrics historically

2. The CLI Setup & Execution Loop

The Red Agent shouldn't run interactive shells. It needs a headless, script-driven CLI loop managed by a lightweight process runner (like systemd on your P2P node) that triggers automatically whenever a new batch file appears.

The Core Daemon Loop (run_audit.sh)

```

Bash
#!/bin/bash
INCOMING_DIR="/opt/red-agent/incoming"
APPROVED_DIR="/opt/red-agent/audit/approved"
QUARANTINE_DIR="/opt/red-agent/audit/quarantined"

while true do
  if [ "$(ls -A $INCOMING_DIR)" ]; then
    for file in "$INCOMING_DIR"/*.jsonl; do
      echo "[+] Auditing batch: $(basename "$file")"

      # Step 1: Run structural validation via deterministic CLI tool (e.g., jq / python linter)
      python3 -c "import json, sys; [json.loads(line) for line in open('$file')]" 2>/dev/null
      if [ $? -ne 0 ]; then
        echo "[-] Schema violation detected. Moving to quarantine."
        mv "$file" "$QUARANTINE_DIR/"
        continue
      fi

      # Step 2: Pass through the auditing model for logic/safety/hallucination checks
      # (Using a local lightweight model instance on the node)
      audit_result=$(python3 /opt/red-agent/scripts/evaluate_batch.py "$file")

      if [ "$audit_result" == "PASS" ]; then
        mv "$file" "$APPROVED_DIR/"
        echo "[✓] Batch approved and moved to sync queue."
      else
        mv "$file" "$QUARANTINE_DIR/"
        echo "[✗] Batch failed adversarial audit. Quarantined."
      fi
    done
  fi
  sleep 10
done

```

3. Key Design Principles for the Red Agent Setup

- **Fail-Closed Isolation:** If a generated JSONL script throws an unparsable error or

attempts to reference assets outside its schema scope, it doesn't get corrected on the fly—it goes straight to quarantined/.

- **Zero Host-Level Exposure:** The Red Agent node should execute inside an isolated container boundary or locked system user scope on your peer-to-peer server so an anomalous agent loop cannot compromise the underlying machine.
- **Asynchronous Handoff:** The UI/Scripting Hub drops payloads into incoming/ via Tailscale SCP or API push, and walks away. The Red Agent processes asynchronously, preventing network lockups across your mesh network.